

When Server-Level Load Imbalance Helps: Power Savings without Capacity Loss

Sherif Abdelrazek*, Dhyey Thummar*, Ahmed Saeed

Georgia Institute of Technology

sabdelrazek3@gatech.edu, dthummar3@gatech.edu, asaeed@cc.gatech.edu

*Student authors

Datacenters are becoming one of the most consequential electricity loads, making energy efficiency a gating factor for cost, siting, and reliability. The 2024 United States datacenter energy usage report [9] estimates that they consumed roughly 176 TWh in 2023 (about 4.4% of U.S. electricity) and projects ~325–580 TWh by 2028. At these scales, energy becomes a first-order economic and infrastructure constraint for datacenters, where even single-digit efficiency gains translate into large recurring savings and determine where and how fast new capacity can be deployed.

A range of approaches have been proposed to improve datacenter energy efficiency, including per-server power management through Dynamic Voltage and Frequency Scaling (DVFS) and power capping [1, 10], energy-aware core scheduling [6], application-level optimizations [13], and spatiotemporal workload shifting [7, 11]. However, these techniques face practical limits. For instance, DVFS can be suboptimal for some workloads, as it reduces CPU computational capacity to lower power consumption. Such decisions can increase the latency of some applications or reduce the effective capacity of machines [10]. Further, the benefits of spatiotemporal shifting can be constrained by limited variability of carbon intensity across time/regions as well as capacity/latency limits [11]. *These limitations motivate the need to explore complementary methods to improve datacenter energy efficiency.*

Motivation. Our proposal exploits two well-known properties of datacenter workloads and hardware. First, the inherent trade-off between utilization and tail latency leads operators to provision excess capacity to satisfy latency SLOs, resulting in substantial idle “slack” to absorb demand bursts [4, 8]. For example, Google reports 23–48% idle capacity [8]. Second, CPUs are not *power-proportional*: at $< 10\%$ utilization, servers can draw $> 25\%$ of peak power [1]; moreover, power scales sublinearly with load due to hardware and runtime effects [3, 12]. Together, these properties imply that for applications provisioned with substantial slack, total energy is governed not only by aggregate load but by how that load is distributed across machines. In particular, balancing the load perfectly across all machines can be power-suboptimal.

Proposal. We propose *load imbalancing*: concentrating work on a subset of machines while leaving others idle or eligible for low-power states. Since power scales sublinearly with load, the additional power consumed by increasing utilization on active machines is outweighed by the power savings from reducing load on the remaining machines. Practically, we can implement imbalancing with weighted round-robin (WRR): designate a subset of machines as *active* with non-zero weights and mark the remainder *idle* with zero weight. To meet latency SLOs, the load balancer temporarily routes requests to idle machines when active machines approach overload (e.g., when requests-in-flight exceed a configured threshold). Thus, imbalancing improves energy efficiency without reducing peak compute capacity, unlike DVFS, which reduces the effective compute capacity of a server. It also differs from “right-sizing” which can be implemented using autoscalers to perfectly match allocated capacity to load. In particular, our proposal never deallocates machines, keeping idle capacity ready to absorb bursts, whereas autoscalers incur provisioning delays before additional capacity becomes available.

Experimental results. We demonstrate the benefits of load imbalancing using a real application deployment. We run a replicated memcached service on a 20-node cluster using the ETC workload; we observe similar trends when using USR [2]. All servers are CloudLab c220g1 machines, and requests arrive according to a Poisson process. We define server capacity (100% load) based on the maximum sustainable throughput at which the 99th-percentile latency reaches $10\times$ the median latency at low load. We assume an aggregate slack capacity of 20%, corresponding to an 80% cluster-wide load and allowing up to four nodes to be idle. We evaluate three imbalancing configurations: (i) 90% load on 16 machines, 80% on 2 machines, and 2 idle; (ii) 95% load on 16 machines, 80% on 1 machine, and 3 idle; and (iii) 100% load on 16 machines with 4 idle. We compare load imbalancing against two baselines in which load is evenly balanced across all machines and power is managed using DVFS. Specifically, we evaluate the performance governor, which pins CPUs at the highest frequency, and `schedutil`, which scales CPU

	Schedutil	Performance	Imbalancing Config 1	Imbalancing Config 2	Imbalancing Config 3
					
	(80% on 20)	(80% on 20)	(90% on 16, 80% on 2, 2 idle)	(95% on 16, 80% on 1, 3 idle)	(100% on 16, 4 idle)
Total CPU Power (W)	934.7	936.8	900.2	891.8	874.6
Median Latency (μ s)	126	124	122	127	159
99% Tail Latency (μ s)	217	206	231	298	746

Table 1: The top row shows the per-machine load distribution for the 20-node memcached cluster under different configurations. Imbalancing reduces total CPU power by **3.7%**, **4.6%**, and **6.4%** for the three configurations considered, respectively, while maintaining tail latency SLO and causing insignificant increase in median latency. Both frequency governors perform similarly because `schedutil` uses the maximum CPU frequency when load is at or above 80% of CPU capacity.

frequency proportionally to load.

Table 1 summarizes our results. Overall, the results support our central hypothesis: *load imbalancing can substantially lower power consumption without significantly harming tail latency*. For instance, compared to the `schedutil` baseline, imbalancing reduces total CPU power by **3.7%**, **4.6%**, and **6.4%** for the three configurations. While throughput is unchanged and median latency remains similar across configurations, the tail-latency cost of these power savings steepens as utilization increases: +6%, +37%, and up to $3.4\times$ at the most aggressive setting. Thus, aggressive imbalancing must be paired with tight overload control to prevent tail-latency spikes [5].

Implications of imbalancing. While these results demonstrate the potential of load imbalancing, our current implementation, based on WRR, is limited to static workloads. Specifically, our evaluation considers fixed weight configurations and a static partitioning of machines into active and idle sets. Such static policies are inherently suboptimal under dynamic load. For example, if WRR is configured for an expected load of 80% but the actual load drops to 60%, excess machines remain active, wasting energy. Conversely, if the load increases to 90%, the static configuration forces the active machines to operate near saturation before the idle machines are utilized, thereby increasing tail latency. These limitations highlight the need for adaptive mechanisms that dynamically identify active and idle machines, preferentially route load to active machines, and seamlessly incorporate idle machines when demand exceeds active capacity.

Another practical consideration for load imbalancing is its reliance on sublinear load–power scaling. While we observe this behavior in several applications, it is not universal. However, even when sublinear scaling does not hold, idle power is typically lower than the power consumed at very low utilization. Consequently, realizing energy savings from load imbalancing in such cases requires keeping machines designated as idle truly idle, rather than lightly loaded. More broadly, the effectiveness of load imbalancing depends on application-specific behavior and the underlying CPU platform, and must be evaluated in that context.

References

- [1] Esmail Asyabi, Azer Bestavros, Erfan Sharafzadeh, and Timothy Zhu. Peafowl: In-application CPU scheduling to reduce power consumption of in-memory key-value stores. In *Proc. 11th ACM Symp. Cloud Comput. (SoCC '20)*, pages 150–164, 2020.
- [2] Berk Atikoglu, Yuehai Xu, Eitan Frachtenberg, Song Jiang, and Mike Paleczny. Workload analysis of a large-scale key-value store. In *Proc. 12th ACM SIGMETRICS/PERFORMANCE Joint Int. Conf. Meas. Model. Comput. Syst. (SIGMETRICS '12)*, pages 53–64, 2012.
- [3] Luiz André Barroso and Urs Hözl. The case for energy-proportional computing. *Computer*, 40(12):33–37, 2007.
- [4] Noman Bashir, Nan Deng, Krzysztof Rzadca, David Irwin, Sree Kodak, and Rohit Jnagal. Take it to the limit: Peak prediction-driven resource overcommitment in datacenters. In *Proc. 16th Eur. Conf. Comput. Syst. (EuroSys '21)*, pages 556–573, 2021.
- [5] Inho Cho et al. Overload control for μ s-scale RPCs with Breakwater. In *Proc. 14th USENIX Symp. Oper. Syst. Des. Implement. (OSDI '20)*, pages 299–314, 2020.
- [6] Md E. Haque, Yuxiong He, Sameh Elnikety, Thu D. Nguyen, Ricardo Bianchini, and Kathryn S. McKinley. Exploiting heterogeneity for tail latency and energy efficiency. In *Proc. 50th Annu. IEEE/ACM Int. Symp. Microarchitecture (MICRO-50)*, pages 625–638, 2017.
- [7] Diptyarop Maji, Kang Yang, Prashant Shenoy, Ramesh K. Sitaraman, and Mani Srivastava. CarbonX: An open-source tool for computational decarbonization using time series foundation models. arXiv preprint, 2025. arXiv:2510.01521.
- [8] Krzysztof Rzadca et al. Autopilot: Workload autoscaling at Google. In *Proc. 15th Eur. Conf. Comput. Syst. (EuroSys '20)*, pages 1–16, 2020.
- [9] Arman Shehabi et al. 2024 United States Data Center Energy Usage Report. December 2024.
- [10] Jovan Stojkovic, Nikoleta Iliakopoulou, Tianyin Xu, Hubertus Franke, and Josep Torrellas. EcoFaaS: Rethinking the design of serverless environments for energy efficiency. In *Proc. ACM/IEEE Int. Symp. Comput. Archit. (ISCA '24)*, pages 471–486, 2024.
- [11] Thanathorn Sukprasert, Abel Souza, Noman Bashir, David Irwin, and Prashant Shenoy. On the limitations of carbon-aware temporal and spatial workload shifting in the cloud. In *Proc. 19th Eur. Conf. Comput. Syst. (EuroSys '24)*, pages 924–941, 2024.
- [12] Hannes Tröpgen, Robert Schöne, Thomas Ilsche, and Daniel Hackenberg. 16 years of SPEC power: An analysis of X86 energy efficiency trends. In *Proc. IEEE Int. Conf. Cluster Comput. Workshops (CLUSTERW '24)*, pages 76–80, 2024.
- [13] Yefu Wang and Xiaorui Wang. Virtual Batching: Request batching for server energy conservation in virtualized data centers. *IEEE Trans. Parallel Distrib. Syst.*, 24(8):1695–1705, 2013.